Router Accuracy: the Confusion Matrix, Worked Out by Hand

Why one "86%" number hides a starved specialist —
every cell, every precision, every recall counted for one 500-query router.

What this document does. It takes the one tiny evaluation from Module 3.2 — the *router* that classifies each customer-support query to one of three specialists (`billing`, `shipping`, `tech`) — fixes a concrete 3×3 confusion matrix over 500 routed queries, and then computes *everything* a routing scorecard reports: per-specialist precision and recall, per-class F1, overall accuracy, and the three F1 averages (macro, micro, weighted). Nothing is hand-waved. Each number below is reproduced by the small Python program in Appendix A, so the arithmetic is exact and self-consistent. The module quotes a router at "overall accuracy 86.4%" that nonetheless leaks a minority class; we rebuild exactly that trap from first principles — our router reads 85.6% overall yet routes only 40% of `tech` queries correctly.

Where this sits. This is **Topic 1 of Module 3.2** — the headline metric behind the "Routing Accuracy" eval surface (surface ① of the six). Its companions: Module 3.1 (single-agent scoring — precision/recall/F1 first appear there) and Topic 2 of this module (why pipeline accuracy compounds — the router's accuracy is the *first* factor in that product).

Concepts covered, in order: routing as multi-class classification · the confusion matrix (true rows, predicted columns) · per-class precision $= M[c,c]/\text{col}(c)$ and recall $= M[c,c]/\text{row}(c)$ · F1 as their harmonic mean · overall accuracy $=$ trace$/N$ · macro-F1 vs. micro-F1 (micro $=$ accuracy) vs. weighted-F1 · why overall accuracy hides a rare, starved specialist.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one router, 500 queries, counted by hand

A router is a classifier in disguise. Each incoming query has a *true* specialist it should go to, and the router emits a *predicted* specialist. Module 3.2 says it plainly: "Routing is a classification problem in disguise. Treat it like one." So we evaluate it exactly as we would any multi-class classifier — by tallying, for every query, the pair (true specialist, predicted specialist) into a grid.

The quantities. We route $N = 500$ labelled queries through a $C = 3$ specialist router. We record the counts in a $C \times C$ *confusion matrix* M, where rows are the *true* specialist and columns are the *predicted* specialist. These counts are illustrative but chosen round so every ratio below is followable on paper; the mechanics are identical at any size.

Quantity	Symbol	Value here
Number of specialists (classes)	C	3 (billing, shipping, tech)
Total labelled queries routed	N	500
Confusion-matrix cell, true i predicted j	$M[i, j]$	see grid below
Support of class c (true count)	$\text{row}(c)$	row sum of M
Predicted count for class c	$\text{col}(c)$	column sum of M
Correct routes to c	$M[c, c]$	the diagonal

The confusion matrix. Rows are the truth; read across a row to see where that specialist's queries actually went. The diagonal (boxed) is correct routing; everything off-diagonal is a misroute.

	predicted →			
true ↓	billing	shipping	tech	**row**(c)
billing	$\boxed{270}$	18	12	300
shipping	20	$\boxed{150}$	10	180
tech	8	4	$\boxed{8}$	20
col(c)	298	172	30	**500**

Notation. For class c, $M[c, c]$ are the true positives. $\text{row}(c)$ is the *support* — how many queries truly belong to c. $\text{col}(c)$ is how many the router *predicted* as c. Everything in the rest of this document is built from these three numbers per class.

> **Intuition**
>
> Read a *row* to ask "of the queries that truly belong to this specialist, where did they go?" — that is recall. Read a *column* to ask "of the queries the router sent to this specialist, how many belonged there?" — that is precision. The matrix holds both questions at once; precision and recall are just the two directions you can slice a single cell.

2 Step 1 — precision and recall, one specialist at a time

Fix a specialist c. Two ratios live in its row and column. **Precision** asks: of everything routed *to* c, what fraction belonged there? It divides the diagonal by the column total. **Recall** asks: of everything that *should* have gone to c, what fraction did? It divides the diagonal by the row total.

$$\text{precision}(c) = \frac{M[c, c]}{\text{col}(c)}, \qquad \text{recall}(c) = \frac{M[c, c]}{\text{row}(c)}. \tag{1}$$

Working all three specialists straight off the grid:

Specialist	$M[c,c]$	**precision** $= M[c,c]/\text{col}(c)$	**recall** $= M[c,c]/\text{row}(c)$
billing	270	$270/298 = 0.9060$	$270/300 = 0.9000$
shipping	150	$150/172 = 0.8721$	$150/180 = 0.8333$
tech	8	$8/30 = 0.2667$	$8/20 = 0.4000$

Sample check, shipping precision: column total $= 20 + 150 + 4 = 172$, and $150/172 = 0.8721$. ✓
Sample check, tech recall: row total $= 8 + 4 + 8 = 20$, and $8/20 = 0.4000$. ✓

Mini-example — this operation in isolation

Look only at `tech`. Its row is $(8, 4, 8)$ — of the 20 queries that truly needed tech, just 8 reached it; the other 12 were misrouted to billing or shipping. So recall $= 8/20 = 0.40$: **the router silently drops three out of every five tech queries**. Its column is $(12, 10, 8)$ — of the 30 queries sent to tech, only 8 belonged there, so precision $= 8/30 = 0.267$. Tech is both *starved* (low recall) and *polluted* (low precision). Neither fact is visible from a single accuracy number; both are sitting in one row and one column.

3 Step 2 — F1, the harmonic mean of the two

Precision and recall pull in opposite directions, so we summarise each class with their *harmonic* mean — the F1 score. The harmonic mean punishes imbalance: if either number is small, F1 stays small.

$$\boxed{F1(c) = \frac{2\,P(c)\,R(c)}{P(c) + R(c)}} \tag{2}$$

Working the three specialists:

Specialist	P	R	$F1 = 2PR/(P+R)$	support
billing	0.9060	0.9000	0.9030	300
shipping	0.8721	0.8333	0.8523	180
tech	0.2667	0.4000	0.3200	20

Sample check, tech F1: $2 \cdot 0.2667 \cdot 0.4000 = 0.21336$; denominator $0.2667 + 0.4000 = 0.6667$; $0.21336/0.6667 = 0.3200$. ✓ Notice F1 sits *between* precision and recall but nearer the smaller of the two — that is the harmonic mean doing its job. Tech's F1 of 0.32 is the loud alarm that neither 0.856 accuracy nor a glance at the diagonal would ever sound.

4 Step 3 — overall accuracy is the trace over the total

The single "how good is the router?" number is *overall accuracy*: the fraction of all queries placed correctly. Correct placements are exactly the diagonal of M, so accuracy is the *trace* (sum of the diagonal) divided by N:

$$\text{accuracy} \;=\; \frac{\operatorname{tr}(M)}{N} \;=\; \frac{\sum_c M[c,c]}{N}. \tag{3}$$

For our grid, $\operatorname{tr}(M) = 270 + 150 + 8 = 428$, so

$$\text{accuracy} \;=\; \frac{428}{500} \;=\; 0.856 \;=\; 85.6\%.$$

Watch out

85.6% looks like a healthy router — and it is a *lie of aggregation*. The number is dominated by the two big classes: billing (300 queries) and shipping (180) together are 96% of the traffic, and the router handles them well, so they alone drag accuracy up near 0.86. The 20 tech queries — where the router only manages 40% recall — are too few to move the average. **A single accuracy figure averages your best specialist and your worst, weighted by how often each is asked. The rare specialist gets starved in silence.** This is precisely the trap the module flags: "Overall 86% looks healthy — but …always inspect the minority class."

5 Step 4 — three ways to average F1: macro, micro, weighted

One F1 per class is honest but unwieldy. Three standard summaries collapse them to a single number — and they disagree on purpose, because each answers a different question.

Macro-F1 is the plain mean of the per-class F1s. Every specialist counts equally, no matter how rare:

$$\text{macro-F1} \;=\; \frac{1}{C}\sum_c F1(c) \;=\; \frac{0.9030 + 0.8523 + 0.3200}{3} \;=\; \frac{2.0753}{3} \;=\; 0.6918.$$

Micro-F1 pools the true positives, false positives, and false negatives across all classes *before* computing one precision and recall. Pooled $\text{TP} = \sum_c M[c,c] = 428$. Every misroute is, from the pooled view, one false positive for the column it landed in and one false negative for the row it left — so $\text{FP} = \text{FN} = N - \text{TP} = 72$. Then

$$\text{micro-}P = \frac{428}{428+72} = 0.856, \quad \text{micro-}R = \frac{428}{428+72} = 0.856, \quad \text{micro-F1} = 0.856.$$

In single-label classification micro-P, micro-R, micro-F1 and accuracy are all the *same number* — every query contributes exactly one TP or one (FP, FN) pair. ✓ That is why a report that shows "micro-F1" and "accuracy" as two columns is showing you the same figure twice.

Weighted-F1 averages the per-class F1s, but weights each by its support fraction $\text{row}(c)/N$ — restoring the class imbalance that macro deliberately removed:

$$\text{weighted-F1} = \sum_c \frac{\text{row}(c)}{N} F1(c) = \tfrac{300}{500}(0.9030) + \tfrac{180}{500}(0.8523) + \tfrac{20}{500}(0.3200) = 0.8614.$$

Average	Value	What it answers
Overall accuracy	0.856	fraction of queries placed right
Micro-F1	0.856	same as accuracy (single-label)
Weighted-F1	0.8614	per-class F1, weighted by how often asked
Macro-F1	0.6918	per-class F1, every specialist counts equally

Intuition

The *gap between macro and the rest is the whole point.* Accuracy, micro-F1, and weighted-F1 all sit near 0.86 because they let the frequent classes vote with their volume. Macro-F1 collapses to 0.69 because it gives the tiny, badly served `tech` class an equal vote — and tech's 0.32 drags the unweighted mean down hard. **When macro-F1 sits far below accuracy, a minority class is being starved.** The size of that gap, $0.856 - 0.692 = 0.16$ here, is a direct read-out of how unequal your specialists' service is.

6 The matrix, drawn

pred→	billing	shipping	tech
billing	270	18	12
shipping	20	150	10
tech	8	4	8

tech recall $= 8/20 = 0.40$

Green cells are the diagonal — correct routes (their sum, 428, over 500 is the accuracy). Orange cells are misroutes. The `tech` row holds only 8 green out of 20: most of that specialist's queries leaked sideways into billing and shipping, which is the low-recall starvation the averages disagree about.

7 Router-metrics cheat-sheet

Confusion cell	$M[i, j] = \#\{\text{true } i,\ \text{predicted } j\}$
Precision of class c	$P(c) = M[c, c] \,/\, \text{col}(c)$
Recall of class c	$R(c) = M[c, c] \,/\, \text{row}(c)$
F1 of class c	$F1(c) = 2P(c)R(c) \,/\, (P(c) + R(c))$
Overall accuracy	$\text{tr}(M)/N = \sum_c M[c, c] \,/\, N$
Macro-F1	$\frac{1}{C} \sum_c F1(c)$ (each class equal)
Micro-F1	pooled TP/FP/FN = accuracy (single-label)
Weighted-F1	$\sum_c (\text{row}(c)/N)\, F1(c)$
Starvation flag	accuracy $\gg$ macro-F1 $\Rightarrow$ a minority class is failing

8 An honest word about these numbers

The cell counts here $(270, 150, 8, \dots)$ are illustrative round numbers, picked so every ratio is followable by hand — exactly as the module quotes a stylised 86% router to make the same point. Your real matrix will have different counts, more classes (the module adds an out-of-domain `reject` class as a fourth row, where the trap is even sharper), and noisier ratios. What is *not* illustrative is the structure: overall accuracy is always $\text{tr}(M)/N$; precision and recall are always the two directions of a diagonal cell; micro-F1 always equals accuracy in single-label

routing; and macro-F1 always exposes the class your traffic-weighted averages hide. Change the counts and the decimals move; the fact that **a frequent-class average can mask a starved specialist** does not. That is the thing to carry into every routing scorecard — and it is why Module 3.2 insists you always inspect the minority class.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the per-class precision/recall/F1 table, the overall accuracy, and the three F1 averages; change the matrix `M` to score your own router.

```python
# router_confusion.py -- reproduces every number in "Router Accuracy, by Hand".
labels = ["billing", "shipping", "tech"]
# M[true][pred]: rows = TRUE specialist, cols = PREDICTED specialist
M = [
    [270, 18, 12], # true billing (support 300)
    [ 20, 150, 10], # true shipping (support 180)
    [ 8, 4, 8], # true tech (support 20) <- rare, starved class
]
C = len(M)
N = sum(sum(row) for row in M)

rowsum = [sum(M[i]) for i in range(C)] # support (true count)
colsum = [sum(M[i][j] for i in range(C)) for j in range(C)] # predicted count

# overall accuracy = trace / total
trace = sum(M[c][c] for c in range(C))
acc = trace / N

prec, rec, f1 = [], [], []
for c in range(C):
    p = M[c][c] / colsum[c]
    r = M[c][c] / rowsum[c]
    f = 2 * p * r / (p + r) if (p + r) else 0.0
    prec.append(p); rec.append(r); f1.append(f)
    print(f"{labels[c]:9s} P={p:.4f} R={r:.4f} F1={f:.4f} support={rowsum[c]}")

macro = sum(f1) / C # each class equal
TP = trace
FP = FN = N - TP # single-label pooling
micro = (2 * TP) / (2 * TP + FP + FN) # == accuracy
weighted = sum((rowsum[c] / N) * f1[c] for c in range(C)) # weight by support

print(f"accuracy = {acc:.4f} (trace {trace} / {N})")
print(f"macro-F1 = {macro:.4f}")
print(f"micro-F1 = {micro:.4f} (equals accuracy: {abs(micro-acc) < 1e-12})")
print(f"weighted-F1 = {weighted:.4f}")

# Expected output:
# billing P=0.9060 R=0.9000 F1=0.9030 support=300
# shipping P=0.8721 R=0.8333 F1=0.8523 support=180
# tech P=0.2667 R=0.4000 F1=0.3200 support=20
# accuracy = 0.8560 (trace 428 / 500)
# macro-F1 = 0.6918
# micro-F1 = 0.8560 (equals accuracy: True)
# weighted-F1 = 0.8614
```

— end of document —